



**TECNOLÓGICO
NACIONAL DE MÉXICO**
INSTITUTO TECNOLÓGICO DE TIJUANA



**Instituto Tecnológico de Tijuana, Departamento de Sistemas y Computación
Tijuana, Baja California, México**

Materia:

Patrones de Diseño

Semestre:

8

Unidad:

II

Docente:

Maribel Guerrero Luis

Alumno:

Leonel Martinez Huitron

Número de control:

22210777

Fecha:

Viernes, 17 de octubre del 2025



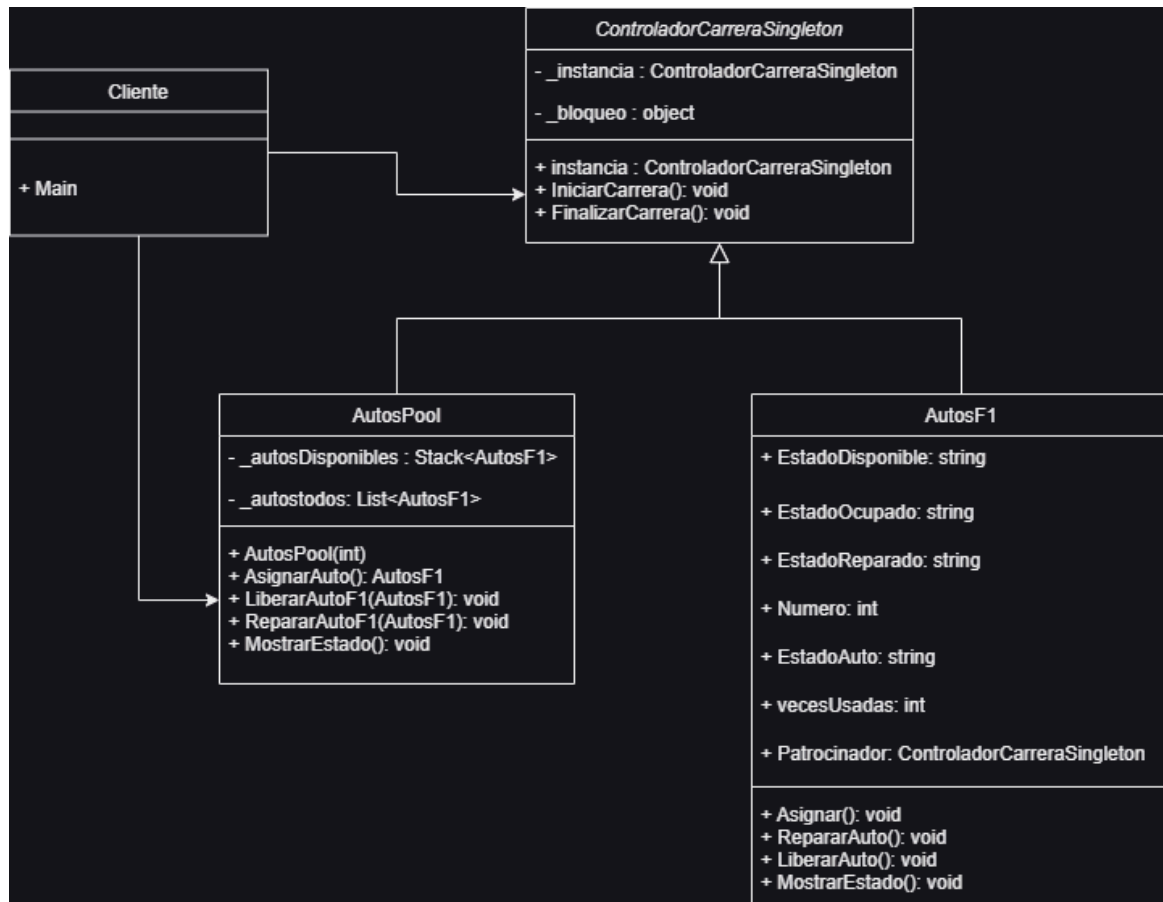


Examen unidad 2

Problema a resolver:

Competencia de autos con obstáculos, aplicando la fusión del patrón Object Pool con Singleton, donde Object Pool debe gestionar objetos reutilizables, mientras que Singleton debe garantizar una sola instancia global.

DIAGRAMA UML





CÓDIGO

```
namespace EXAMEN_U2_ObjectPool_And_Singleton
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            //Tienes un carro de F1, donde tu patron Singleton es el controlador de carrera,
            //en este caso estos controladores de carrera serian los patrocinadores de dichos competidores de cada carro/auto de formula 1.
            //Mientras que el object pool lo que hace es si tenemos un vehiculo o auto de formula 1 que choca/accidenta,
            //dicho vehiculo vuelve a la piscina de objetos, y vuelve a estar disponible otro vehiculo/auto, donde se puede reutilizar.

            Console.WriteLine("=== COMPETENCIA DE AUTOS ===\n");
            //Creacion del Object Pool con 3 autos disponibles
            AutosPool pool = new AutosPool(3);
            Console.WriteLine("Autos creados: (3 disponibles)\n");
            //Asignar autos del pool
            var auto1 = pool.AsignarAuto();
            var auto2 = pool.AsignarAuto();
            var auto3 = pool.AsignarAuto();

            //Mostrar los patrocinadores de cada auto
            Console.WriteLine($"Auto {auto1.Numero} patrocinado por Controlador de Carrera: {auto1.Patrocinador.GetHashCode()}");
            Console.WriteLine($"Auto {auto2.Numero} patrocinado por Controlador de Carrera: {auto2.Patrocinador.GetHashCode()}");
            Console.WriteLine($"Auto {auto3.Numero} patrocinado por Controlador de Carrera: {auto3.Patrocinador.GetHashCode()}");

            //Validar que todos los autos comparten el mismo patrocinador (Singleton)
            bool mismaInstancia = ReferenceEquals(auto1.Patrocinador, auto2.Patrocinador) &&
                ReferenceEquals(auto2.Patrocinador, auto3.Patrocinador) &&
                ReferenceEquals(auto1.Patrocinador, ControladorCarreraSingleton.instancia);

            Console.WriteLine($"Todos los autos comparten la misma instancia de patrocinador? {mismaInstancia}");

            auto1.Patrocinador.IniciarCarrera();
            pool.MostrarEstado();

            pool.LiberarAutoF1(auto3);
            pool.MostrarEstado();

            pool.RepararAutoF1(auto3);
            pool.MostrarEstado();

            auto3 = pool.AsignarAuto();
            pool.MostrarEstado();

            auto1.Patrocinador.FinalizarCarrera();

            Console.WriteLine("\n=== COMPETENCIA DE AUTOS ===\n");
            Console.ReadKey();
        }
    }
}
```



```
namespace EXAMEN_U2_ObjectPool_And_Singleton
{
    7 referencias
    public class ControladorCarreraSingleton
    {
        //Propiedad para obtener la instancia del singleton
        private static ControladorCarreraSingleton _Instancia;
        //Objeto para el bloqueo en la creacion de la instancia
        private static readonly object _bloqueo = new object();

        1 referencia
        private ControladorCarreraSingleton()
        {
        }

        //Propiedad estatica para obtener la instancia unica del singleton
        2 referencias
        public static ControladorCarreraSingleton instancia
        {
            get
            {
                if(_Instancia == null)
                {
                    lock (_bloqueo)
                    {
                        if(_Instancia == null)
                        {
                            _Instancia = new ControladorCarreraSingleton();
                        }
                    }
                }
                return _Instancia;
            }
        }

        1 referencia
        public void IniciarCarrera()
        {
            Console.WriteLine($"La carrera ha comenzado");
            Console.WriteLine();
        }

        1 referencia
        public void FinalizarCarrera()
        {
            Console.WriteLine();
            Console.WriteLine($"La carrera ha terminado");
        }
    }
}
```



```
namespace EXAMEN_U2_ObjectPool_And_Singleton
{
    11 referencias
    public class AutosF1
    {
        //Propiedad para obtener la instancia del singleton
        11 referencias
        public ControladorCarreraSingleton Patrocinador { get; private set; }
        public const string EstadoDisponible = "Disponible";
        public const string EstadoOcupado = "En Carrera";
        public const string EstadoReparado = "En Mantenimiento Pendiente";
        public readonly int Numero;
        7 referencias
        public string EstadoAuto { get; private set; }
        private int vecesUsadas = 0;
        //Constructor de la clase AutosF1
        1 referencia
        public AutosF1 (int numero)
        {
            Numero = numero;
            EstadoAuto = EstadoDisponible;
            Patrocinador = ControladorCarreraSingleton.instancia;
        }
        //Metodo para asignar el auto
        1 referencia
        public void Asignar()
        {
            EstadoAuto = EstadoOcupado;
            vecesUsadas++;
        }
        //Metodo para liberar el auto
        1 referencia
        public void RepararAuto()
        {
            EstadoAuto = EstadoDisponible;
        }
        //Metodo para reparar el auto
        1 referencia
        public void LiberarAuto()
        {
            EstadoAuto = EstadoReparado;
        }
        1 referencia
        public void MostrarEstado()
        {
            Console.WriteLine($"Autos {Numero}: {EstadoAuto} | Usada {vecesUsadas} veces");
        }
    }
}
```



```
namespace EXAMEN_U2_ObjectPool_And_Singleton
{
    3 referencias
    public class AutosPool
    {
        private readonly Stack<AutosF1> _autosDisponibles = new Stack<AutosF1>();
        private readonly List<AutosF1> _autostodos = new List<AutosF1>();

        //Constructor del pool de autos
        1 referencia
        public AutosPool(int cantidadInicial)
        {
            for(int i = 1; i <= cantidadInicial; i++)
            {
                var auto = new AutosF1(i);
                _autosDisponibles.Push(auto);
                _autostodos.Add(auto);
                Console.WriteLine($"Total de Autos en el pool: {_autosDisponibles.Count}");
            }
        }

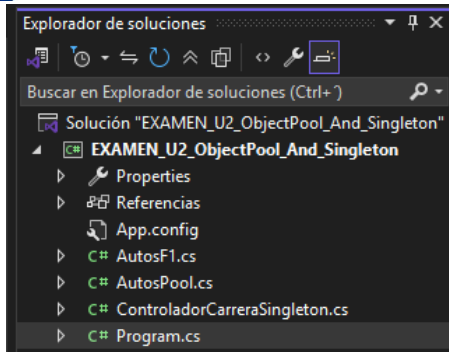
        //Metodo para asignar un auto del pool
        4 referencias
        public AutosF1 AsignarAuto()
        {
            if (_autosDisponibles.Count > 0)
            {
                var auto = _autosDisponibles.Pop();
                auto.Asignar();
                Console.WriteLine($"Auto: {auto.Numero} asignado.");
                return auto;
            }

            Console.WriteLine("No hay autos disponibles en el pool.");
            return null;
        }

        //Metodo para liberar un auto del pool
        1 referencia
        public void LiberarAutoF1(AutosF1 auto)
        {
            if (auto.EstadoAuto == AutosF1.EstadoOcupado)
            {
                auto.LiberarAuto();
                Console.WriteLine($"Auto: {auto.Numero} liberado y en mantenimiento.");
            }
            else
            {
                Console.WriteLine($"Auto {auto.Numero} no puede ser liberado porque no está en carrera.");
            }
        }
    }
}
```

```
    //Metodo para reparar un auto del pool
    1 referencia
    public void RepararAutoF1(AutosF1 auto)
    {
        if (auto.EstadoAuto == AutosF1.EstadoReparado)
        {
            auto.RepararAuto();
            _autosDisponibles.Push(auto);
            Console.WriteLine($"Auto: {auto.Numero} reparado y disponible en el pool.");
        }
        else
        {
            Console.WriteLine($"Auto {auto.Numero} no está en mantenimiento, no se puede reparar.");
        }
    }

    4 referencias
    public void MostrarEstado()
    {
        Console.WriteLine($"-----Estado del Pool-----");
        foreach(var carros in _autostodos)
        {
            carros.MostrarEstado();
        }
        Console.WriteLine();
    }
}
```

SALIDA POR CONSOLA

```
C:\Users\Layonel\source\repc X + v

=== COMPETENCIA DE AUTOS ===

Total de Autos en el pool: 1
Total de Autos en el pool: 2
Total de Autos en el pool: 3
Autos creados: (3 disponibles)

Auto: 3 asignado.
Auto: 2 asignado.
Auto: 1 asignado.
Auto 3 patrocinado por Controlador de Carrera: 46104728
Auto 2 patrocinado por Controlador de Carrera: 46104728
Auto 1 patrocinado por Controlador de Carrera: 46104728

¿Todos los autos comparten la misma instancia de patrocinador? True
La carrera ha comenzado

-----Estado del Pool-----
Autos 1: En Carrera | Usada 1 veces
Autos 2: En Carrera | Usada 1 veces
Autos 3: En Carrera | Usada 1 veces

Auto: 1 liberado y en mantenimiento.
-----Estado del Pool-----
Autos 1: En Mantenimiento Pendiente | Usada 1 veces
Autos 2: En Carrera | Usada 1 veces
Autos 3: En Carrera | Usada 1 veces

Auto: 1 reparado y disponible en el pool.
-----Estado del Pool-----
Autos 1: Disponible | Usada 1 veces
Autos 2: En Carrera | Usada 1 veces
Autos 3: En Carrera | Usada 1 veces

Auto: 1 asignado.
-----Estado del Pool-----
Autos 1: En Carrera | Usada 2 veces
Autos 2: En Carrera | Usada 1 veces
Autos 3: En Carrera | Usada 1 veces

La carrera ha terminado

=== COMPETENCIA DE AUTOS ===
```



CONCLUSIÓN

En conclusión, la integración del patrón Singleton y Object Pool en un mismo sistema puede ofrecer tanto eficiencia y reutilización siempre y cuando su implementación sea en base a los principios de estos patrones, es decir, el saber la separación de responsabilidades, en este caso, el patrón Singleton se debe encargar de la lógica de control global, así como su validación de dicha instancia, mientras que Object Pool debe gestionar la creación, asignación y reciclaje de objetos. Personalmente considero que el patrón Singleton coordina, toma decisiones globales y mantiene el control central del sistema basándose en una única entidad o instancia, y el Object Pool se da el lujo de reutilizar, asignar, liberar y manteniéndolos según las necesidades que requiera el sistema. Esto a nivel práctico nos ofrece buenas prácticas como evitar el acoplamiento innecesario, reutilización segura de objetos así como el control de concurrencias (verificaciones).